



SMART CONTRACT AUDIT REPORT

for

SymbioticProxy



Prepared By: Xiaomi Huang

PeckShield
June 18, 2026

Document Properties

Client	BedRock
Title	Smart Contract Audit Report
Target	SymbioticProxy
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	June 18, 2026	Xuxian Jiang	Final Release
1.0-rc	June 18, 2026	Xuxian Jiang	Release Candidate

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1	Introduction	4
1.1	About SymbioticProxy	4
1.2	About PeckShield	5
1.3	Methodology	5
1.4	Disclaimer	7
2	Findings	9
2.1	Summary	9
2.2	Key Findings	10
3	Detailed Results	11
3.1	Suggested Immutable Use in SymbioticProxy	11
3.2	Revisited maxRewards Count in claimRewards()	12
4	Conclusion	14
	References	15

1 | Introduction

Given the opportunity to review the design document and related source code of the `SymbioticProxy` smart contract, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About SymbioticProxy

`Symbiotic` is a permissionless, modular restaking protocol that allows networks to secure themselves using existing assets. The audited `SymbioticProxy` contract manages the `cuniBTC` vault to aggregate user deposits, deploy them into external yield strategies, and distribute returns proportionally within an epoch-based framework. Strategy execution may be supported by AI-assisted systems, while all fund flows are governed on-chain. The basic information of audited contracts is as follows:

Table 1.1: Basic Information of Audited Contracts

Item	Description
Target	<code>SymbioticProxy</code>
Website	https://bedrock.technology/
Type	EVM Smart Contract
Language	Solidity
Audit Method	Whitebox
Latest Audit Report	June 18, 2026

In the following, we show the Git repository of reviewed files and the commit hash values used in this audit. The given repository contains a number of contracts and this audit only covers the following contract, i.e., `SymbioticProxy.sol`.

- <https://github.com/Bedrock-Technology/uniBTC.git> (db99ccf)

And this is the commit ID after all fixes for the issues found in the audit have been checked in:

- <https://github.com/Bedrock-Technology/uniBTC.git> (34ae3b8)

1.2 About PeckShield

PeckShield Inc. [5] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/peckshield>), Twitter (<http://twitter.com/peckshield>), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low
		High	Medium	Low
		Likelihood		

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [3], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the implementation of the `SymbioticProxy` smart contract. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logic, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	0	
Low	0	
Informational	2	
Total	2	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in [Section 3](#).

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 2 informational recommendations.

Table 2.1: Key SymbioticProxy Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Informational	Suggested Immutable Use in SymbioticProxy	Coding Practices	Resolved
PVE-002	Informational	Revisited maxRewards Count in claimRewards()	Coding Practices	Resolved

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to Section 3 for details.



3 | Detailed Results

3.1 Suggested Immutable Use in SymbioticProxy

- ID: PVE-001
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: SymbioticProxy
- Category: Coding Practices [2]
- CWE subcategory: CWE-561 [1]

Description

Since version 0.6.5, [Solidity](#) introduces the feature of declaring a state as `immutable`. An `immutable` state variable can only be assigned during contract creation, but will remain constant throughout the life-time of a deployed contract. The main benefit of declaring a state as `immutable` is that reading the state is significantly cheaper than reading from regular storage, since it is not stored in storage anymore. Instead, an `immutable` state will be directly inserted into the runtime code.

This feature is introduced based on the observation that the reading and writing of storage-based contract states are gas-expensive. Therefore, it is always preferred if we can reduce, if not eliminate, storage reading and writing as much as possible. Those state variables that are written only once are candidates of `immutable` states under the condition that each fits the pattern, i.e., “a constant, once assigned in the constructor, is read-only during the subsequent operation.”

While examining all the state variables defined in the `SymbioticProxy` smart contract, we observe there are several variables that need not to be updated dynamically. They can be declared as `immutable` for gas efficiency.

```
12 contract SymbioticProxy is Ownable, ReentrancyGuard {  
13     address public symbioticVault;  
14     address public defaultStakerRewards;  
15     address public vault;  
16     address public uniBTC;
```

Listing 3.1: SymbioticProxy

Recommendation Revisit the state variable definition and make good use of `immutable` states for the following storage states, i.e., `symbioticVault`, `defaultStakerRewards`, `vault`, and `uniBTC`.

Status The issue has been addressed by the following commit: `a8b042e`.

3.2 Revisited maxRewards Count in claimRewards()

- ID: PVE-002
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: `SymbioticProxy`
- Category: Coding Practices [2]
- CWE subcategory: CWE-561 [1]

Description

In [Solidity](#), it is considered best practice to limit loop iterations so that we can prevent permanent gas-related Denial of Service (DoS) vulnerabilities and to control unpredictable transaction costs. This is imposed largely by the fact that smart contracts run on a decentralized network and code execution is tightly constrained by the Ethereum Virtual Machine (EVM). While reviewing the `SymbioticProxy` contract, we notice a large loop iteration limit is unnecessary and can be revised to be significantly smaller.

To elaborate, we show below the implementation of the related `claimRewards()` routine. As the name indicates, this routine is used to claim rewards from the deposited assets. It comes to our attention that an internal variable of `maxRewards` (line 78) that defines the maximum number of reward tokens that may be returned. Considering a typical `USDC` token at `0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48` and `SymbioticNetwork` at `0x98e52Ea7578F2088c152E81b17A9a459bF089f2a`, the query of current reward tokens returns only 7, which indicates the current loop limit of `maxRewards` as 1000 (line 78) can be significantly smaller.

```

74     function claimRewards(address network, address rewardToken) external onlyOwner
       nonReentrant {
75         require(rewardToken != address(0), "SymbioticProxy: invalid reward token");

77         bytes[] memory activeSharesOfHints = new bytes[](0);
78         uint256 maxRewards = 1000;
79         bytes memory rewardData = abi.encode(network, maxRewards, activeSharesOfHints);
80         bytes memory callData = abi.encodeWithSelector(
81             IDefaultStakerRewards.claimRewards.selector,
82             vault,
83             rewardToken,
84             rewardData
85         );

```

```
87     IVault(vault).execute(defaultStakerRewards, callData, 0);  
88 }
```

Listing 3.2: SymbioticProxy::claimRewards()

Recommendation Revisit the above `maxRewards` variable to choose a smaller loop count.

Status This issue has been resolved as the team confirms the `maxRewards` amount is the suggested value from the official Symbiotic protocol.



4 | Conclusion

In this audit, we have analyzed the design and implementation of the `SymbioticProxy` smart contract that manages the existing `cuniBTC` vault to aggregate user deposits, deploy them into external yield strategies, and distribute returns proportionally within an epoch-based framework. Strategy execution may be supported by AI-assisted systems, while all fund flows are governed on-chain. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Meanwhile, we need to emphasize that smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.



References

- [1] MITRE. CWE-561: Dead Code. <https://cwe.mitre.org/data/definitions/561.html>.
- [2] MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- [3] MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.
- [4] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.
- [5] PeckShield. PeckShield Inc. <https://www.peckshield.com>.

